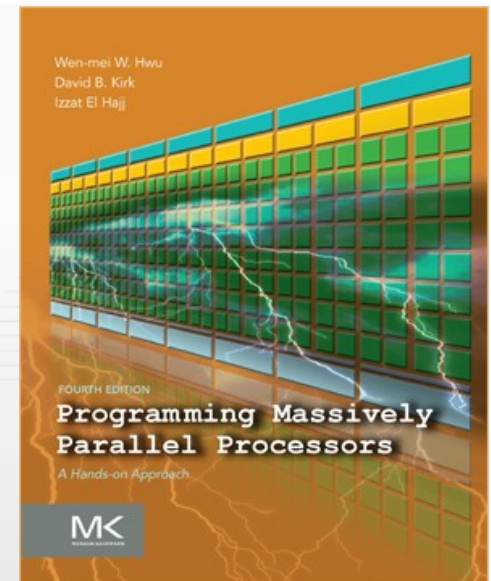


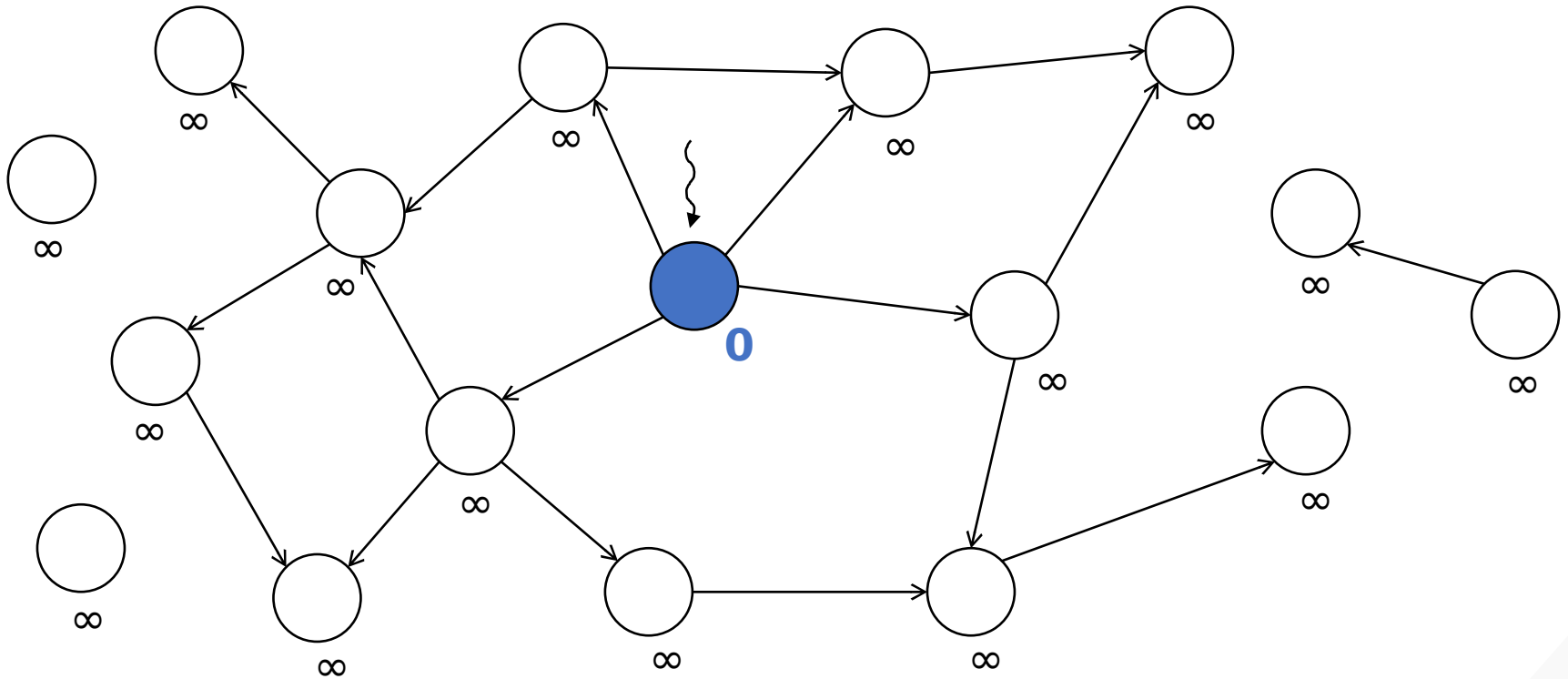
Programming Massively Parallel Processors

A Hands-on Approach

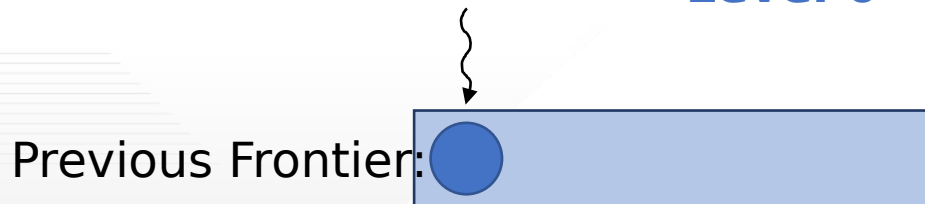
CHAPTER 15 (PART 2) > Graph Traversal

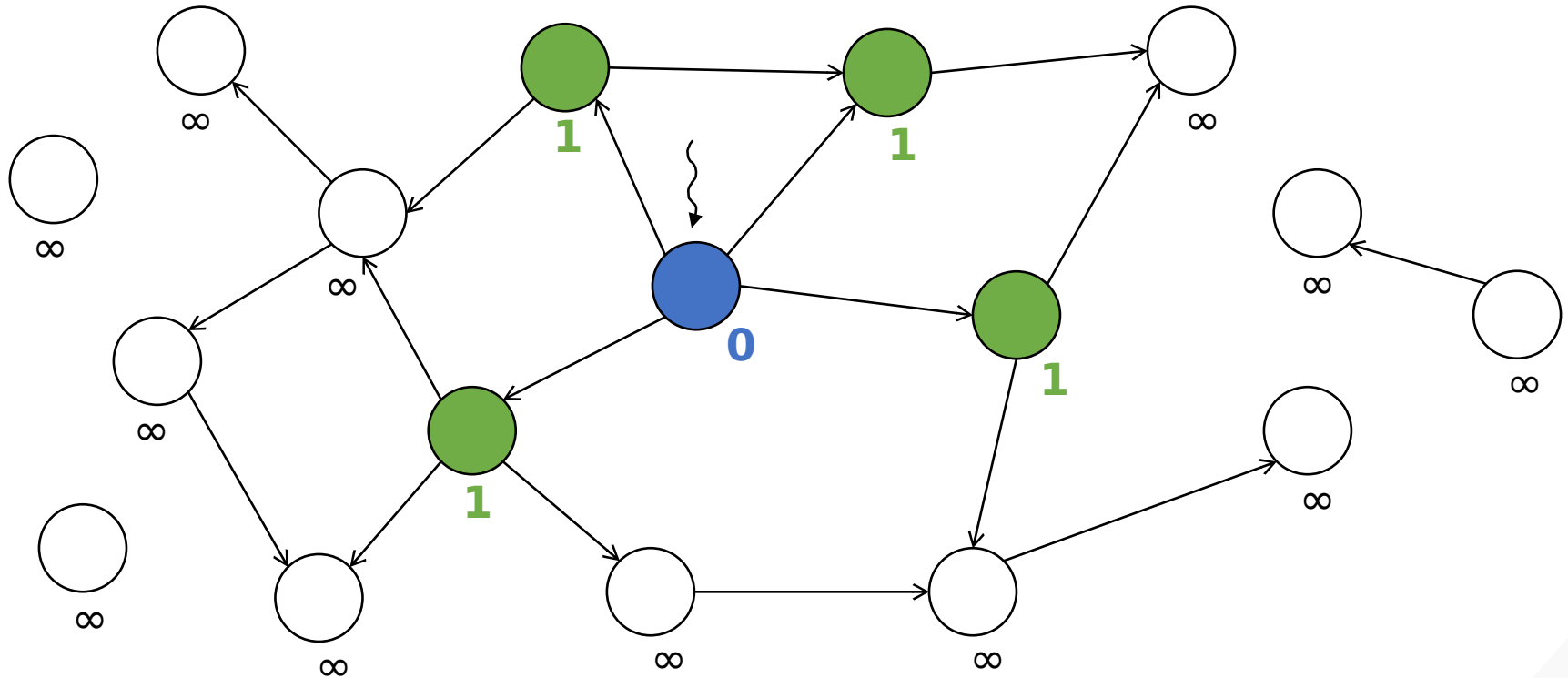


- Approaches discussed so far check every vertex or edge on every iteration for relevance
 - Strengths: easy to implement, highly parallel, no synchronization across threads
 - Weaknesses: a lot of unnecessary threads/work
 - Many threads will find that their vertex or edge are not relevant for this iteration and just exit
- Alternative for reducing redundancy:
 - Objective: only check the vertices that are part of the previous level
 - Approach: each level adds the vertices it visits to a list for the next level to process
 - Vertices added at each level form that level's *frontier*
 - Overhead: synchronization across threads to add to a shared list



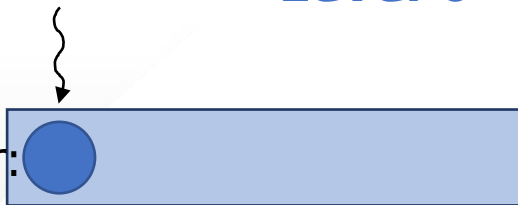
Level 0 => Level 1



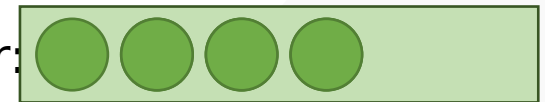


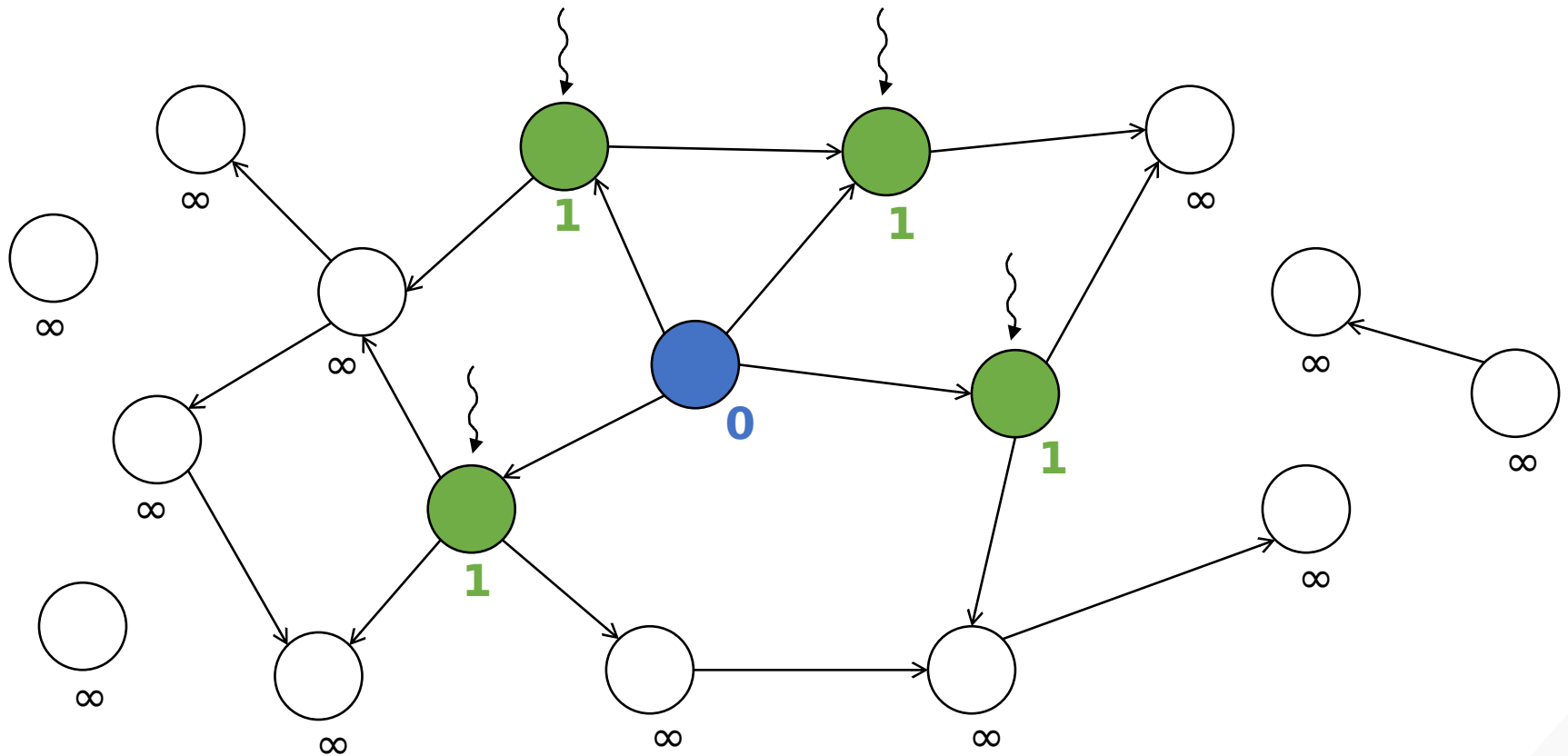
Level 0 => Level 1

Previous Frontier:



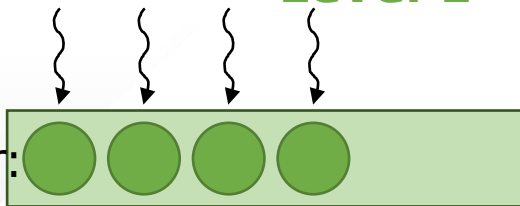
Next Frontier:

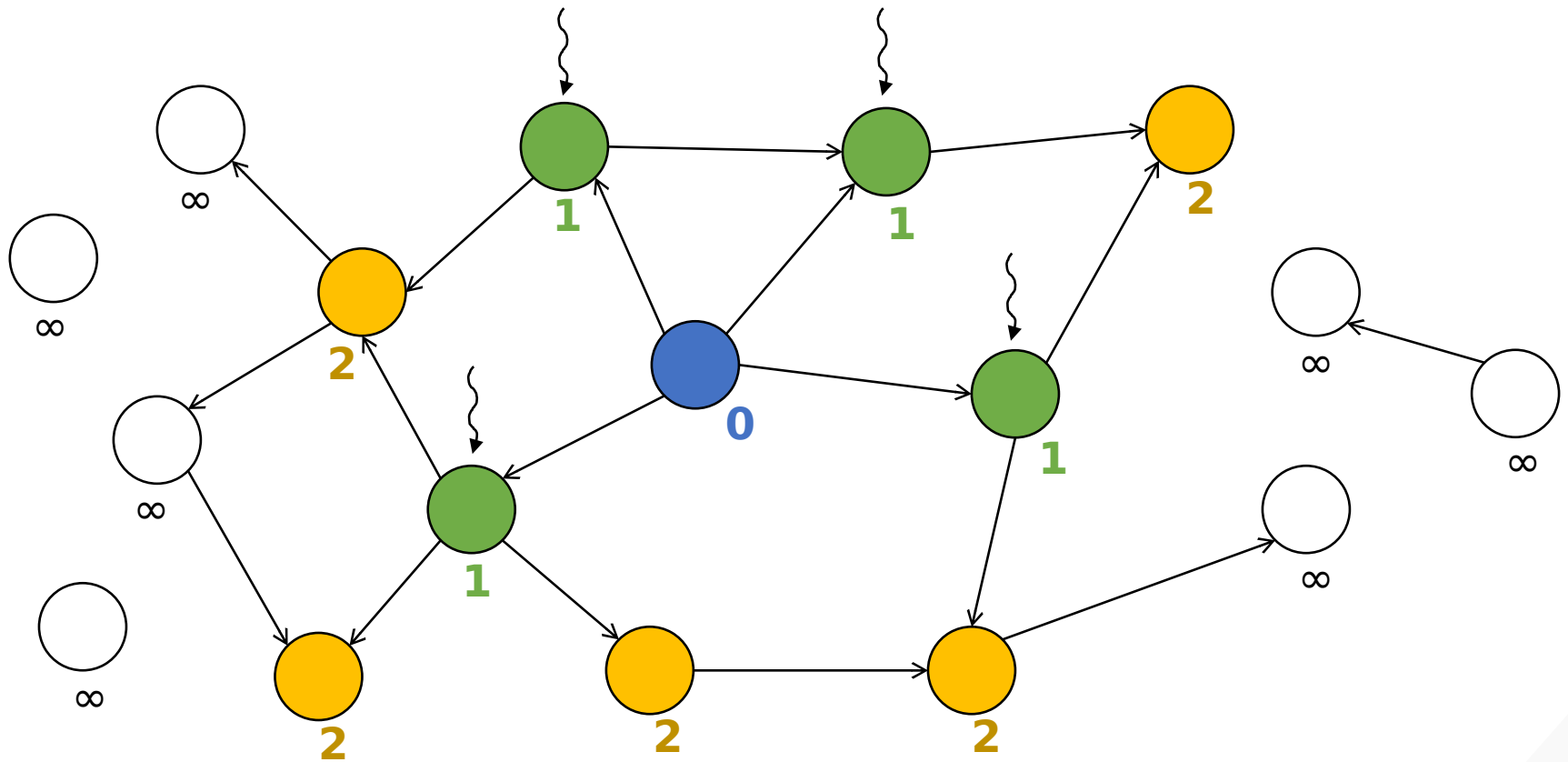




Level 1 $\Rightarrow$ Level 2

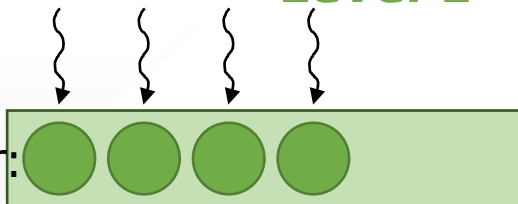
Previous Frontier:



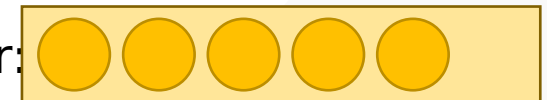


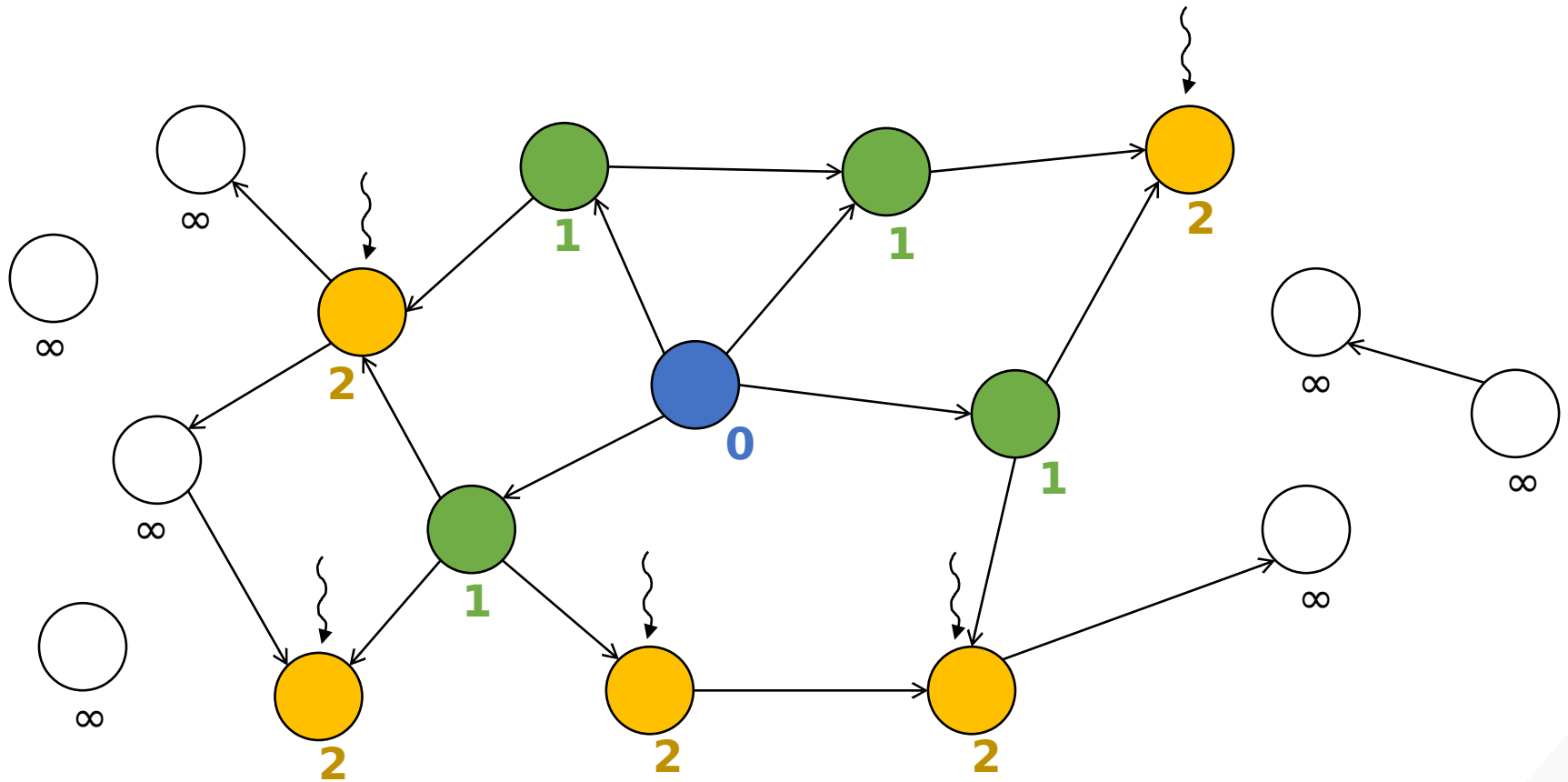
Level 1 => Level 2

Previous Frontier:



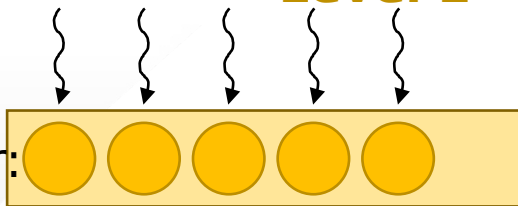
Next Frontier:

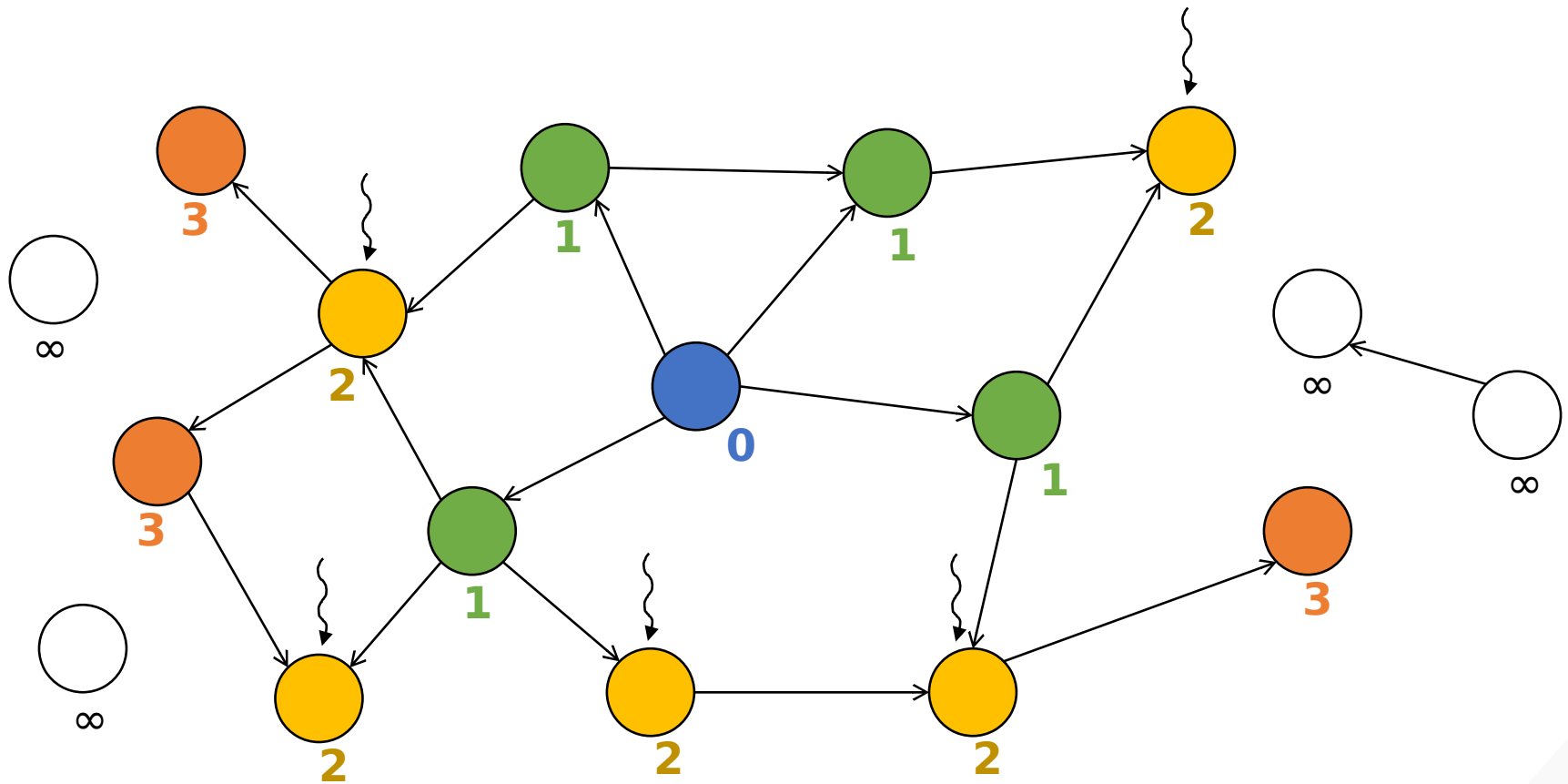




Level 2 => Level 3

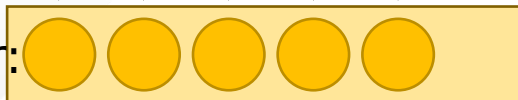
Previous Frontier:



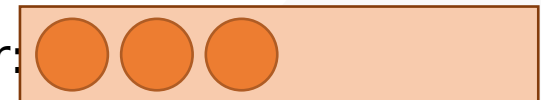


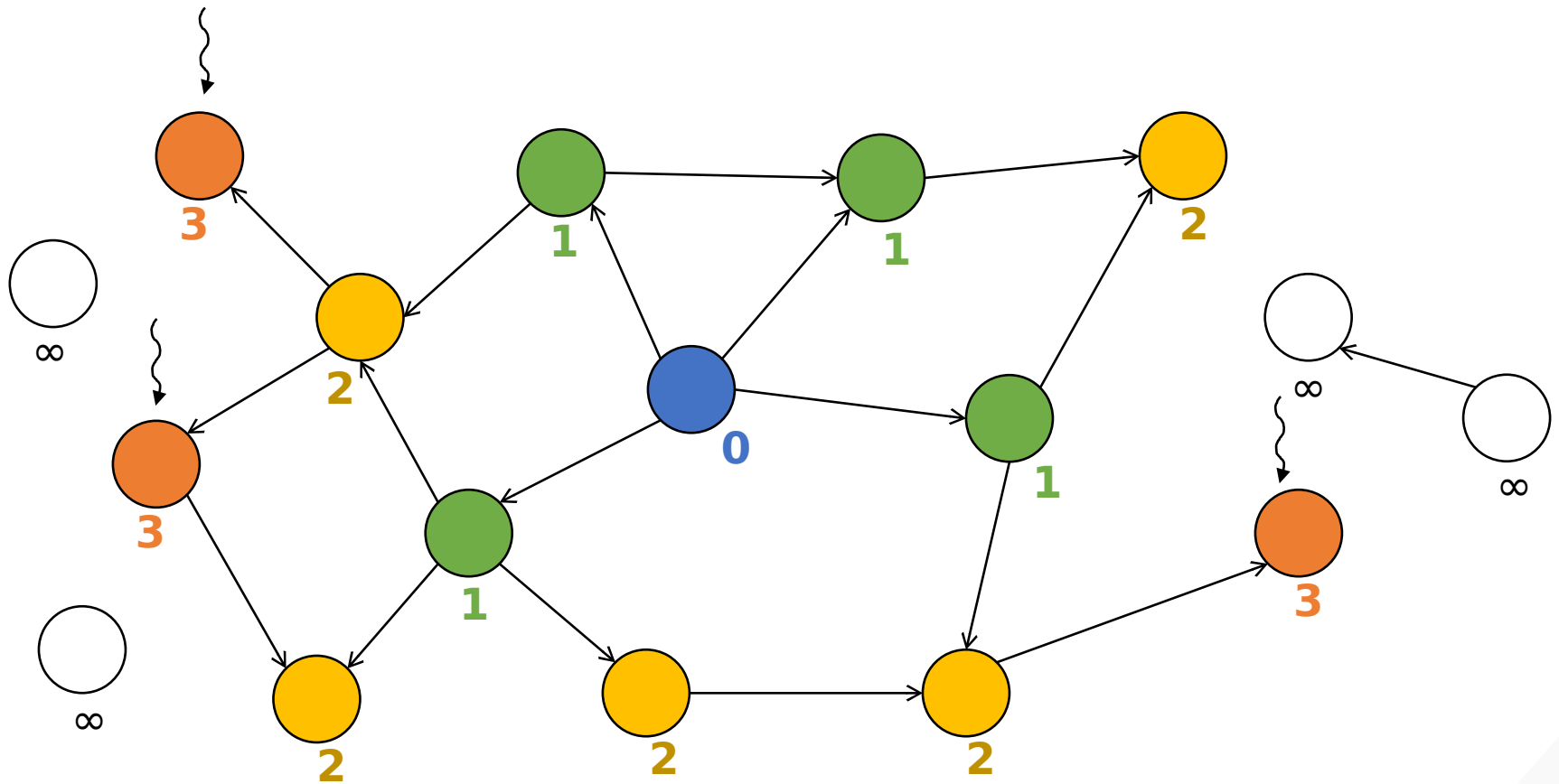
Level 2 => Level 3

Previous Frontier:



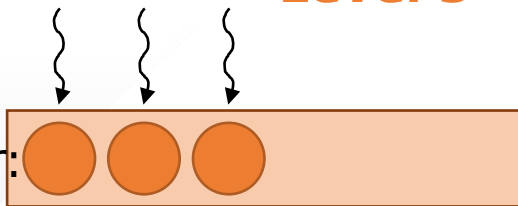
Next Frontier:

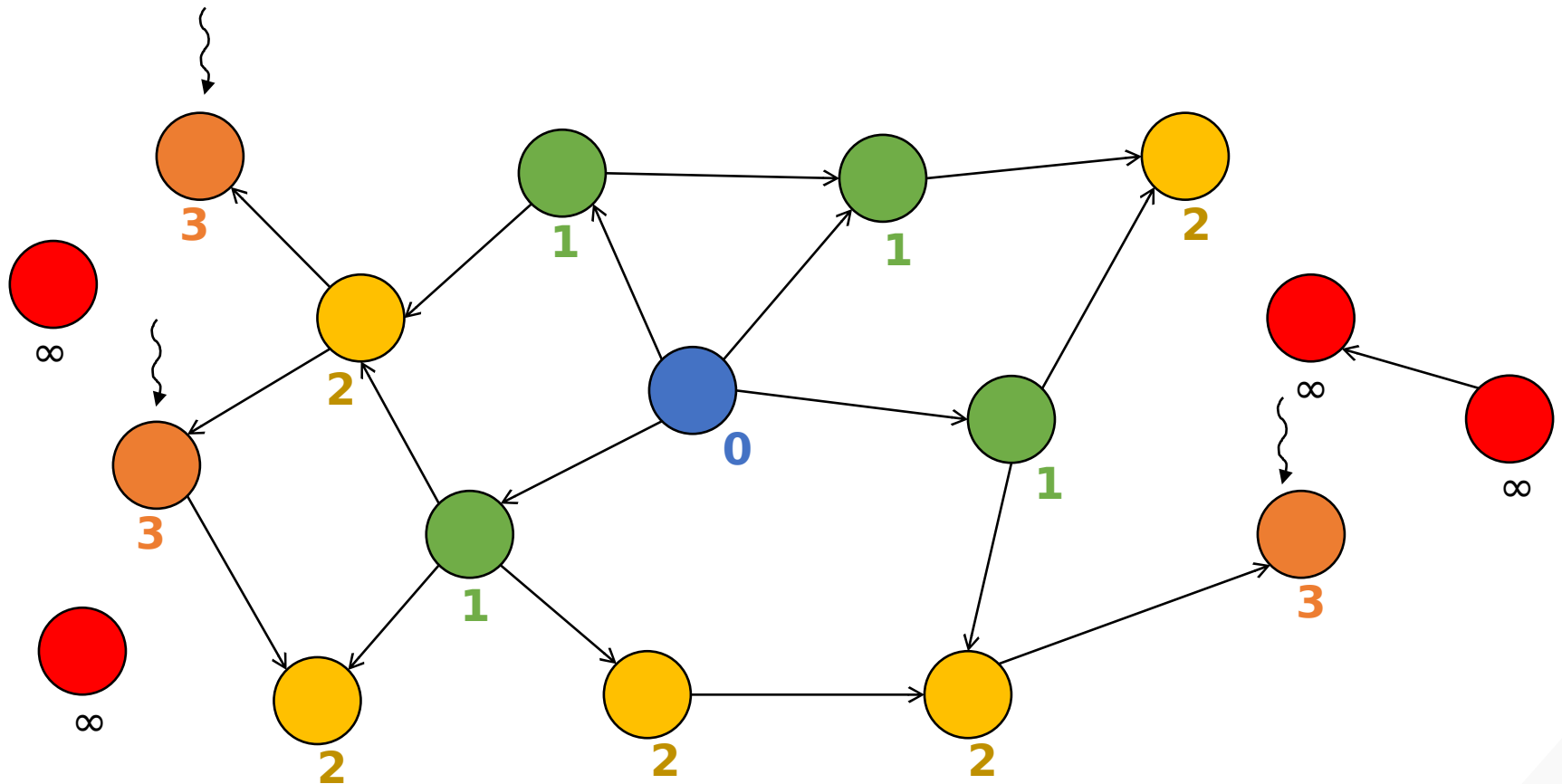




Level 3 => Level 4

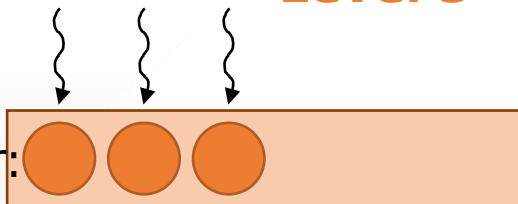
Previous Frontier:





Level 3 => Level 4

Previous Frontier:



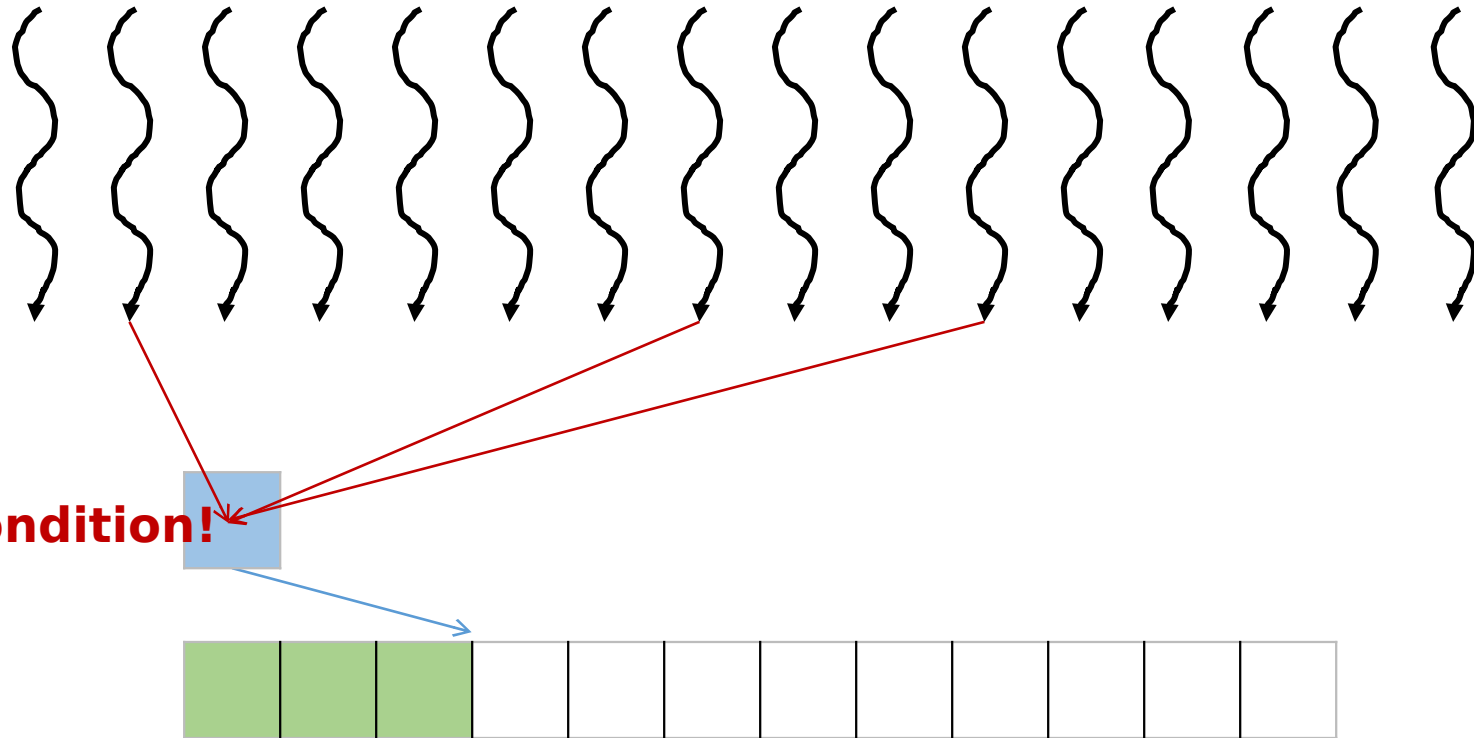
Next Frontier:

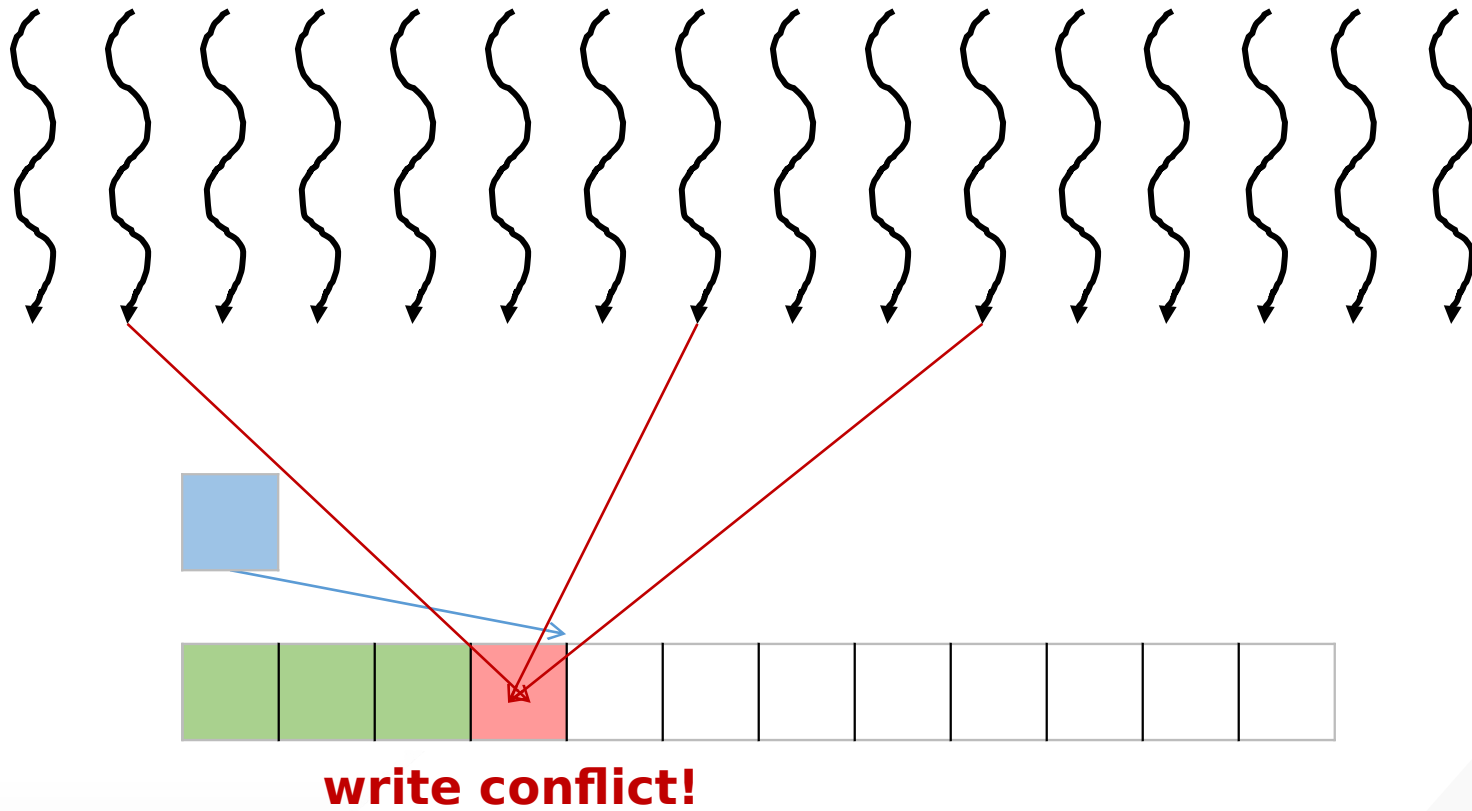


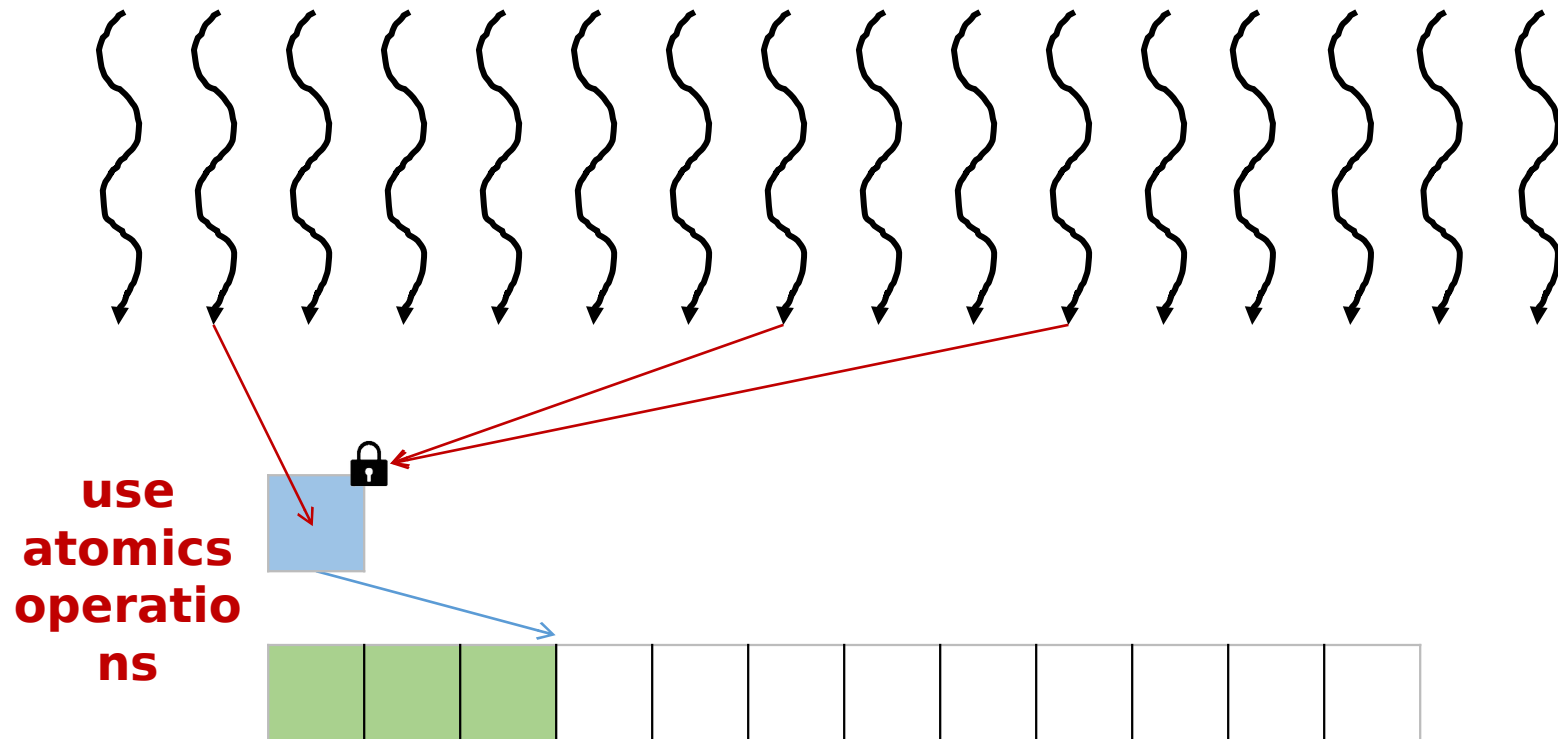
```
__global__ void bfs_kernel(CSRGraph csrGraph, unsigned int* level, unsigned int* prevFrontier,
                           unsigned int* currFrontier, unsigned int numPrevFrontier,
                           unsigned int* numCurrFrontier, unsigned int currLevel) {
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    if(i < numPrevFrontier) {
        unsigned int vertex = prevFrontier[i];
        for(unsigned int edge = csrGraph.srcPtrs[vertex]; edge < csrGraph.srcPtrs[vertex + 1];
            ++edge) {
            unsigned int neighbor = csrGraph.dst[edge];
            if(atomicCAS(&level[neighbor], UINT_MAX, currLevel) == UINT_MAX) {
                unsigned int currFrontierIdx = atomicAdd(numCurrFrontier, 1);
                currFrontier[currFrontierIdx] = neighbor;
            }
        }
    }
}
```

Ensure only one thread visits the neighbor to avoid adding it to the frontier multiple times

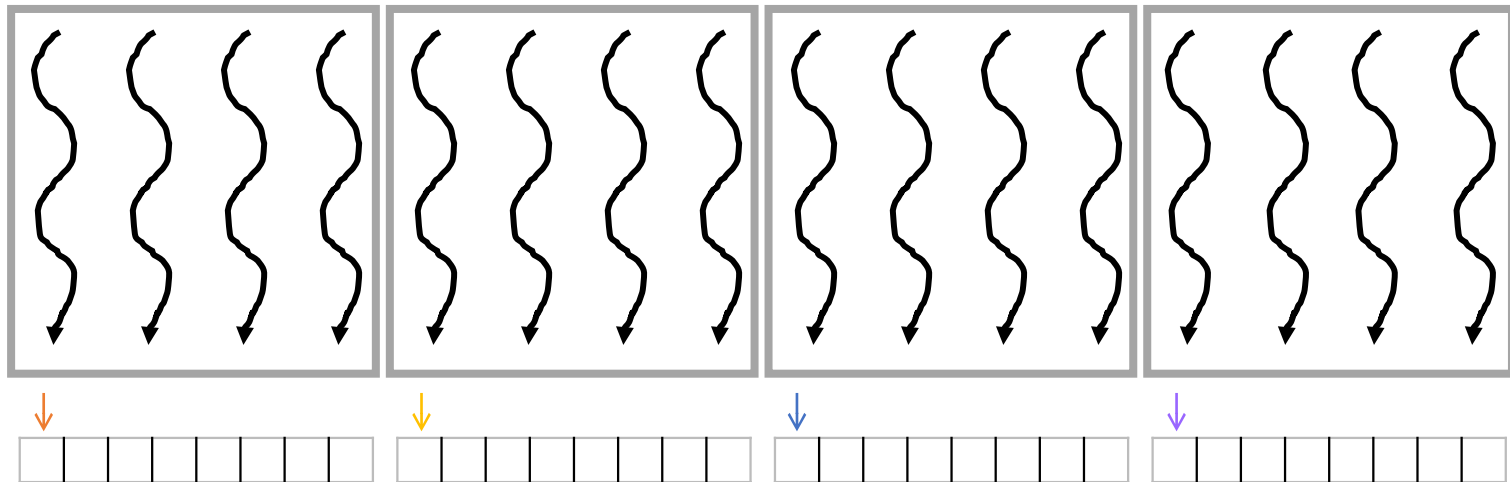
Avoid race conditions when adding to the frontier

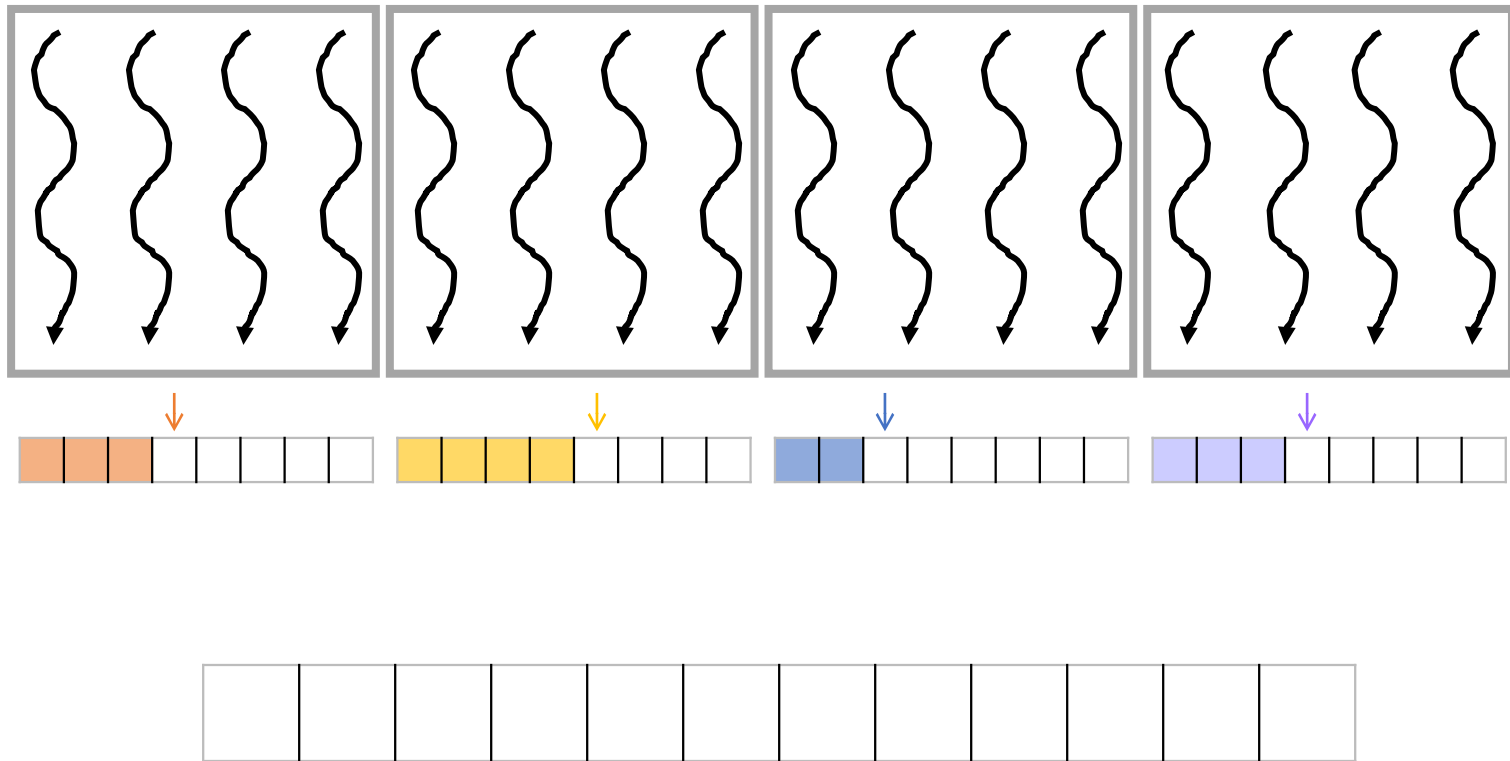


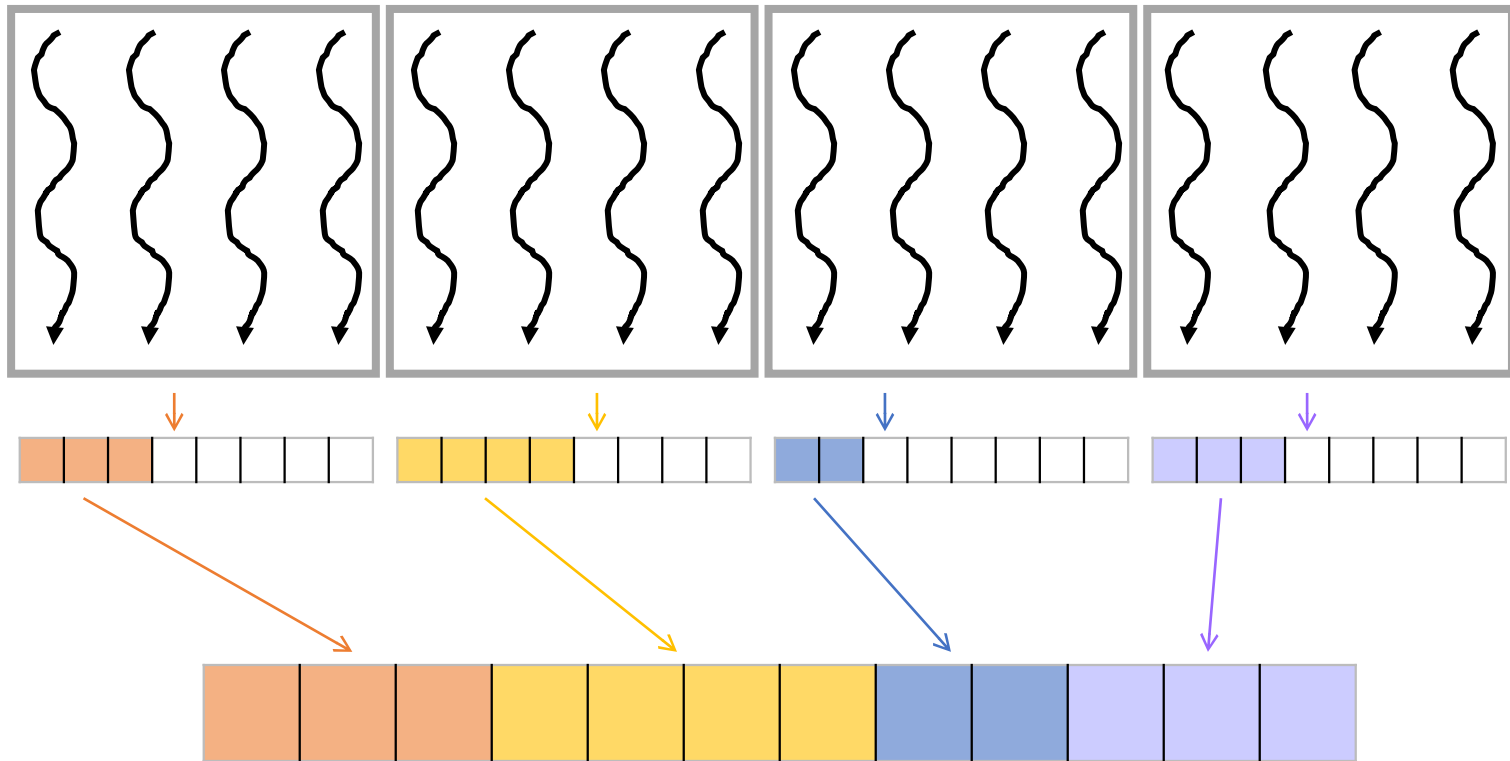




- All threads atomically increment the same global counter to add elements to the frontier
 - High latency due to global memory access and serialization due to high contention
- Optimization: privatization and shared memory
 - Each thread block maintains a private frontier in shared memory and commits entries to the global frontier upon completion







```
__global__ void bfs_kernel(CSRGraph csrGraph, unsigned int* level, unsigned int* prevFrontier,
    unsigned int* currFrontier, unsigned int numPrevFrontier, unsigned int* numCurrFrontier, unsigned int currLevel) {

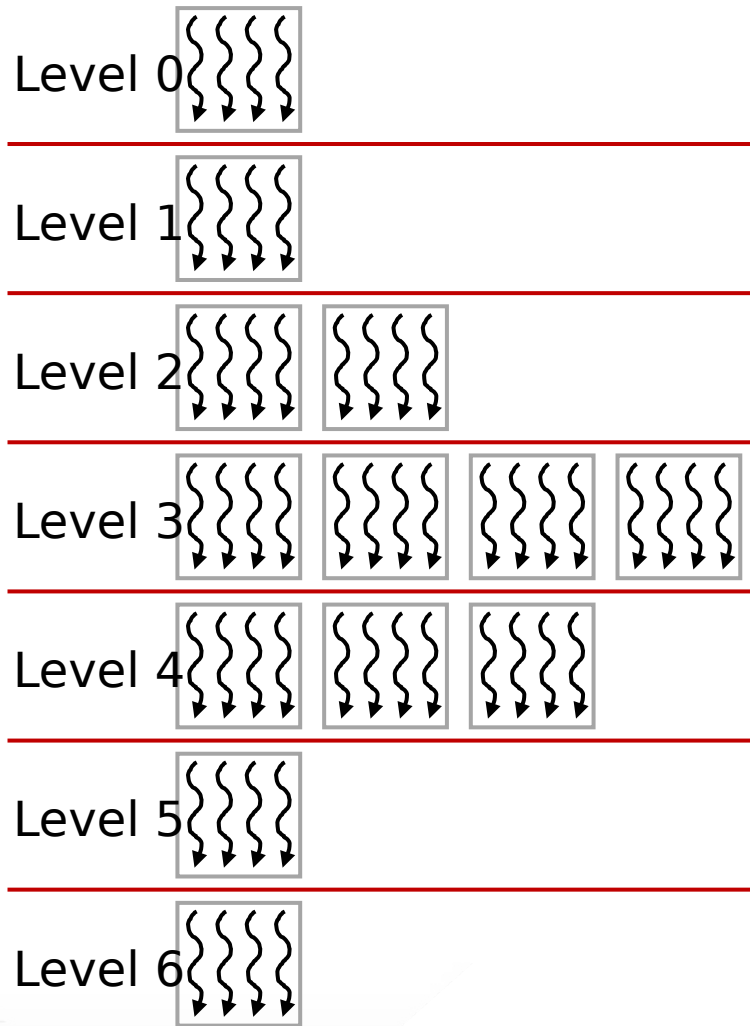
    // Initialize privatized frontier
    __shared__ unsigned int currFrontier_s[LOCAL_FRONTIER_CAPACITY];
    __shared__ unsigned int numCurrFrontier_s;
    if(threadIdx.x == 0) {
        numCurrFrontier_s = 0;
    }
    __syncthreads();

    // Perform BFS
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    if(i < numPrevFrontier) {
        unsigned int vertex = prevFrontier[i];
        for(unsigned int edge = csrGraph.srcPtrs[vertex]; edge < csrGraph.srcPtrs[vertex + 1]; ++edge) {
            unsigned int neighbor = csrGraph.dst[edge];
            if(atomicCAS(&level[neighbor], UINT_MAX, currLevel) == UINT_MAX) { // Vertex not previously visited
                unsigned int currFrontierIdx_s = atomicAdd(&numCurrFrontier_s, 1);
                if(currFrontierIdx_s < LOCAL_FRONTIER_CAPACITY) {
                    currFrontier_s[currFrontierIdx_s] = neighbor;
                } else {
                    numCurrFrontier_s = LOCAL_FRONTIER_CAPACITY;
                    unsigned int currFrontierIdx = atomicAdd(numCurrFrontier, 1);
                    currFrontier[currFrontierIdx] = neighbor;
                }
            }
        }
    }
    __syncthreads();

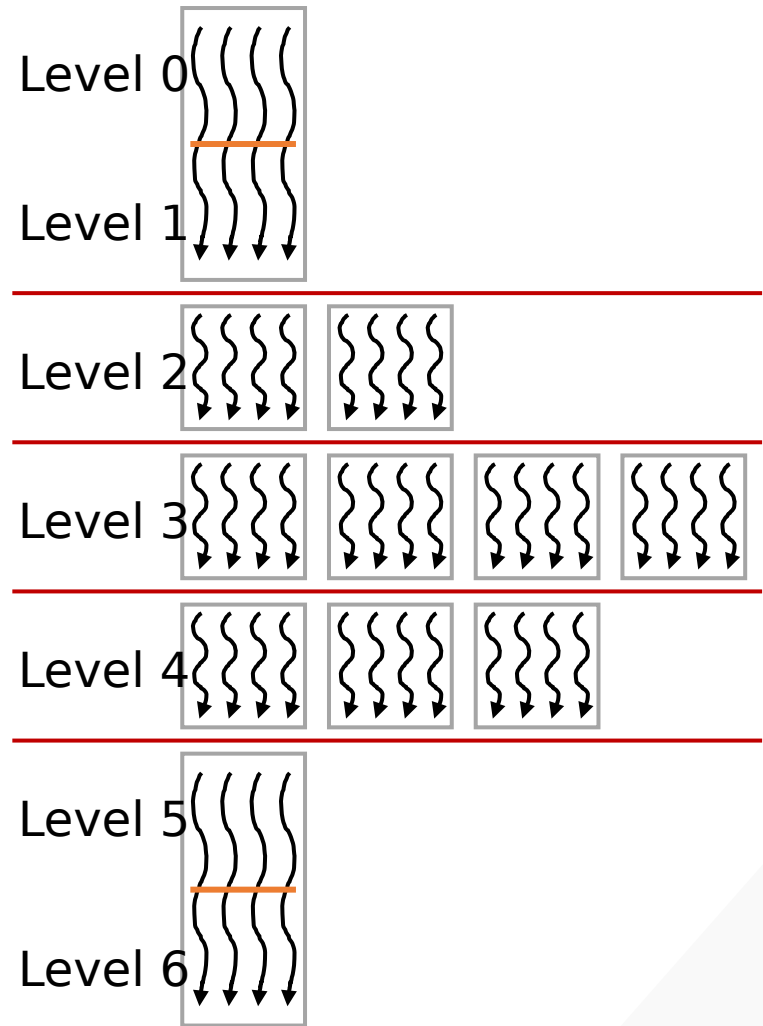
    // Allocate in global frontier
    __shared__ unsigned int currFrontierStartIdx;
    if(threadIdx.x == 0) {
        currFrontierStartIdx = atomicAdd(numCurrFrontier, numCurrFrontier_s);
    }
    __syncthreads();

    // Commit to global frontier
    for(unsigned int currFrontierIdx_s = threadIdx.x; currFrontierIdx_s < numCurrFrontier_s; currFrontierIdx_s += blockDim.x) {
        unsigned int currFrontierIdx = currFrontierStartIdx + currFrontierIdx_s;
        currFrontier[currFrontierIdx] = currFrontier_s[currFrontierIdx_s];
    }
}
```

- Need to synchronize between levels
 - Wait for all threads to add all vertices in the current level to the frontier before proceeding to the next level
- So far, we have launched a new grid for each level
 - Overhead of launching new grid and copying counter
- Optimization: If consecutive levels have few enough vertices that can be executed by one thread block:
 - Execute multiple levels in one single-block grid and synchronize between levels using `__syncthreads()`
 - Reduces total number of number of grid launches



Launching a new grid for
each level



Consecutive small levels in
one grid

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.